

Enterprise DevOps Platform — Final Master Documentation

Objective

Build a complete enterprise-grade local DevOps lab environment focused on:

- Git
- GitHub
- Docker
- Kubernetes
- Jenkins
- Ansible
- Prometheus
- Grafana
- CI/CD Pipelines
- Monitoring
- Container Orchestration

The main objective is NOT Azure cloud deployment.

The primary focus is:

- Containerization
- Kubernetes orchestration
- CI/CD automation
- Infrastructure automation
- Monitoring & observability
- Git-based workflows
- Enterprise DevOps practices

Build a complete enterprise-grade local DevOps lab environment using:

- GitHub
- Docker
- Kubernetes
- Terraform
- Jenkins
- Azure Cloud
- Monitoring Stack
- DevSecOps Tools
- CI/CD Pipelines

The goal is to:

- Create real-world portfolio projects
- Push projects to GitHub
- Build CI/CD pipelines
- Deploy workloads on Kubernetes

- Practice Infrastructure as Code
 - Build Cloud Architect / DevOps Engineer level hands-on skills
-

Enterprise DevOps Platform — Final Objective

Build a complete enterprise-grade DevOps platform locally using:

- Git
- GitHub
- Docker
- Kubernetes
- Jenkins
- Ansible
- Prometheus
- Grafana
- CI/CD Pipelines
- Monitoring & Observability

The complete learning objective is:

```
Code Development
  ↓
Git & GitHub Version Control
  ↓
Docker Containerization
  ↓
Kubernetes Deployment
  ↓
Jenkins CI/CD Automation
  ↓
Ansible Configuration Automation
  ↓
Prometheus Monitoring
  ↓
Grafana Dashboards & Alerts
```

Final Architecture Goal

```
Developer
  ↓
GitHub Repository
  ↓
CI/CD Pipeline (Jenkins / GitHub Actions)
  ↓
```

```
Docker Image Build
↓
Container Registry
↓
Kubernetes Deployment
↓
Monitoring & Security
```

Local Lab Environment

Installed Tools

Tool	Purpose	Status
VS Code	Development IDE	Installed
Docker Desktop	Containers	Installed
Terraform	Infrastructure as Code	Installed
kubectl	Kubernetes management	Installed
kind	Local Kubernetes cluster	Installed
Azure CLI	Azure authentication	Installed
GitHub CLI	GitHub integration	In Progress

PHASE 1 — GitHub & Git Setup

Step 1 — Login to GitHub CLI

Run:

```
& "C:\Program Files\GitHub CLI\gh.exe" auth login
```

Select:

```
GitHub.com
HTTPS
Login with browser
Authenticate Git
```

Verify:

```
& "C:\Program Files\GitHub CLI\gh.exe" auth status
```

Step 2 — Configure Git Identity

```
git config --global user.name "Satish Mali"  
git config --global user.email "YOUR_EMAIL"
```

Verify:

```
git config --global --list
```

PHASE 2 — Enterprise Repository Structure

Create Main Folder

```
mkdir D:\enterprise-cloud-platform  
cd D:\enterprise-cloud-platform
```

Create Folder Structure

```
mkdir DevOps  
mkdir Terraform  
mkdir Security  
mkdir Monitoring  
mkdir Hybrid_Infra  
mkdir Documentation
```

DevOps Projects

```
mkdir DevOps\Docke Projects  
mkdir DevOps\Jenkins_CICD  
mkdir DevOps\Kubernetes  
mkdir DevOps\GitHub_Actions
```

Terraform Projects

```
mkdir Terraform\Azure_Landing_Zone
mkdir Terraform\AKS_Deployment
mkdir Terraform\Multi_Cloud
```

Security Projects

```
mkdir Security\DevSecOps
mkdir Security\Wazuh
mkdir Security\Zero_Trust
```

Monitoring Projects

```
mkdir Monitoring\Prometheus_Grafana
mkdir Monitoring\ELK
```

PHASE 3 — Create GitHub Repository

Create Repository

Repository Name:

```
enterprise-cloud-platform
```

Initialize Git

```
git init
```

Create README.md

```
notepad README.md
```

Add:

```
# Enterprise Cloud Platform

Enterprise-grade DevOps, Cloud, Kubernetes, Terraform, Security, Monitoring,
and Hybrid Infrastructure projects.
```

First Commit

```
git add .
git commit -m "Initial enterprise cloud platform structure"
```

Create GitHub Repo from CLI

```
& "C:\Program Files\GitHub CLI\gh.exe" repo create enterprise-cloud-platform
--public --source=. --remote=origin
```

Push Code

```
git branch -M main
git push -u origin main
```

PHASE 4 — Project 1: Docker Website Deployment

Objective

Create:

- Simple website
 - Docker image
 - GitHub integration
 - Kubernetes deployment
-

Folder

```
DevOps/Docker_Projects/simple-webapp
```

Structure

```
simple-webapp/  
├─ index.html  
├─ style.css  
├─ Dockerfile  
├─ deployment.yaml  
├─ service.yaml  
├─ README.md  
└─ screenshots/
```

Step 1 — Create Website

index.html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Enterprise DevOps Lab</title>  
    <link rel="stylesheet" href="style.css">  
  </head>  
  <body>  
    <h1>Enterprise DevOps Platform</h1>  
    <p>Docker + Kubernetes + Jenkins Project</p>  
  </body>  
</html>
```

Step 2 — Create Dockerfile

```
FROM nginx:latest  
COPY . /usr/share/nginx/html  
EXPOSE 80
```

Step 3 — Build Docker Image

```
docker build -t enterprise-webapp .
```

Step 4 — Run Container

```
docker run -d -p 8080:80 enterprise-webapp
```

Access:

```
http://localhost:8080
```

PHASE 5 — Kubernetes Deployment

Create deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: enterprise-webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: enterprise-webapp
  template:
    metadata:
      labels:
        app: enterprise-webapp
    spec:
      containers:
        - name: enterprise-webapp
          image: enterprise-webapp
          imagePullPolicy: Never
          ports:
            - containerPort: 80
```

Create service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: enterprise-webapp-service
spec:
  type: NodePort
  selector:
    app: enterprise-webapp
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Deploy to Kubernetes

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Verify

```
kubectl get pods
kubectl get svc
```

PHASE 6 — Jenkins CI/CD

Objective

Automate:

- GitHub pull
 - Docker build
 - Kubernetes deployment
-

Jenkins Flow

```
GitHub Push
↓
Jenkins Trigger
↓
Docker Build
↓
Kubernetes Deploy
```

Jenkins Installation (Docker)

```
docker run -d --name jenkins
-p 8081:8080
-p 50000:50000
jenkins/jenkins:lts
```

Access Jenkins

```
http://localhost:8081
```

PHASE 7 — Ansible Automation

Objective

Automate:

- Linux server configuration
- Package installation
- Docker installation
- Kubernetes prerequisites
- Configuration management

Folder

```
DevOps/Ansible/
```

Structure

```
Ansible/  
├─ inventory.ini  
├─ ansible.cfg  
├─ playbooks/  
├─ roles/  
├─ group_vars/  
└─ README.md
```

Real-Life Use Cases

Use Case	Automation
Install Docker	Automated
Configure Linux	Automated
Deploy applications	Automated
Kubernetes preparation	Automated
Service restart	Automated

Example Playbook

```
- hosts: all  
  become: yes  
  
  tasks:  
    - name: Install nginx  
      apt:  
        name: nginx  
        state: present
```

Run Playbook

```
ansible-playbook -i inventory.ini playbooks/install-nginx.yml
```

Enterprise Automation Flow

```
graph TD;
  GitHub --> Jenkins;
  Jenkins --> Ansible;
  Ansible --> DockerKubernetes;
  style DockerKubernetes text:Docker/Kubernetes Deployment
```

PHASE 8 — Monitoring Stack

Objective

Build enterprise-grade observability and monitoring platform for:

- Kubernetes
- Docker
- Linux Servers
- CI/CD Pipelines
- Cloud Infrastructure

Monitoring Tools

Tool	Purpose
Prometheus	Collect metrics and monitoring data
Grafana	Dashboard visualization
AlertManager	Send alerts
Wazuh	Security monitoring
ELK Stack	Log analytics

Prometheus + Grafana Architecture

```
graph TD;
  Apps[Applications / Kubernetes / Servers] --> Prometheus;
  Prometheus["Prometheus  
(Metric Collection)"] --> Grafana;
```

Grafana
(Visualization)
↓
Alerts / Dashboard

What Prometheus Does

Prometheus continuously collects:

- CPU usage
- Memory usage
- Kubernetes pod metrics
- Node metrics
- API response time
- Container metrics
- Network traffic
- Disk usage

Prometheus acts as:

Monitoring Engine + Time-Series Database

What Grafana Does

Grafana connects with Prometheus and creates:

- Dashboards
- Visual graphs
- Monitoring panels
- SLA dashboards
- Alerting views
- Infrastructure reports

Real Enterprise Monitoring Flow

Application
↓
Docker Container
↓
Kubernetes Cluster
↓
Prometheus Metrics Collection
↓

```
Grafana Dashboards
  ↓
AlertManager
  ↓
Email / Slack Alerts
```

Folder Structure

```
Monitoring/
├── Prometheus_Grafana/
│   ├── prometheus.yml
│   ├── grafana-dashboard.json
│   ├── alert-rules.yml
│   ├── docker-compose.yml
│   ├── screenshots/
│   └── README.md
```

Future Monitoring Scope

Component	Monitoring
Kubernetes	Pods/Nodes
Docker	Containers
Jenkins	Pipeline status
Linux Servers	CPU/RAM
Azure Resources	Infra monitoring
Security Logs	Wazuh/ELK

Real Resume Line

Implemented enterprise observability platform using Prometheus and Grafana for Kubernetes, Docker, and infrastructure monitoring with alerting and dashboard visualization.

PHASE 9 — DevSecOps

Tools

- Trivy
- tfsec
- Checkov
- SonarQube
- Snyk

Security Pipeline

```
Code Commit
  ↓
Security Scan
  ↓
Build Validation
  ↓
Deployment
```

Recommended Learning Order

Phase	Topic
1	GitHub + Git
2	Docker
3	Kubernetes
4	Jenkins
5	Terraform
6	Monitoring
7	DevSecOps
8	Multi-Cloud
9	Hybrid Infra

Final GitHub Goal

```
enterprise-cloud-platform/  
|  
├─ DevOps  
├─ Terraform  
├─ Security  
├─ Monitoring  
├─ Hybrid_Infra  
└─ Documentation
```

Final Resume Goal

Projects

Enterprise CI/CD Kubernetes Platform

- Designed CI/CD pipelines using Jenkins, Docker, and Kubernetes
- Automated deployments using Infrastructure as Code
- Integrated monitoring and DevSecOps scanning

Azure Landing Zone Automation

- Built reusable Terraform modules for Azure governance and networking
- Implemented RBAC, storage backend, and monitoring architecture

Final Objective

After completing this lab:

- Strong GitHub portfolio
- Enterprise-level documentation
- Real DevOps projects
- Terraform automation experience
- Kubernetes deployment experience
- CI/CD implementation
- Cloud architecture exposure
- Strong interview preparation